

预习报告

实验题目：简单函数绘图语言词法分析器的设计与实现

一、实验目的

1. 理解词法分析器在编译过程中的核心作用，将源代码字符串分解为词法单元（Token）。
2. 掌握简单函数绘图语言的词法规则，准确划分关键字、标识符、常量、运算符、分隔符。
3. 学会为不同类型的 Token 设计正则表达式，并理解匹配优先级（如关键字优先于标识符）。
4. 使用 Python 语言实现 Lexer 类，具备基本的错误处理能力（如识别非法字符）。

二、实验原理

词法分析是编译的第一阶段，通过扫描源程序字符流，根据预定义的词法规则识别出具有独立语义的最小单位——Token。

核心原理包括：

1. **状态转换**：从初始状态出发，根据读入字符的类型（字母、数字、符号）转移至相应状态。
2. **正则匹配**：利用 re 模块的命名分组功能，一次性匹配多种模式，并根据匹配成功的组名确定 Token 类型。
3. **过滤机制**：在扫描过程中自动跳过空白字符（空格、制制符、换行）和注释。

三、实验要求

1. 单词分类
 - 关键字 (KEYWORD): plot, func, range
 - 标识符 (IDENTIFIER): 以字母开头，由字母或数字组成，如 f1, x, val
 - 常量 (NUMBER):
 - 整数 (INTEGER): 如 123, 0
 - 浮点数 (FLOAT): 如 3.14, .5, 100.
 - 运算符 (OPERATOR): +, -, *, /, ^, =
 - 分隔符 (DELIMITER): (,), ,, ;, [,]
2. 待分析的源程序的输入形式和识别后单词的输出形式

- 输入形式: 字符串形式的 Python 代码段或从文件读取。
- 输出形式: 二元组 (Type, Value) 序列。例如: Token('KEYWORD', 'plot')。

3. 单词状态转换图

- Initial 状态:
 - 遇到字母 -> 进入 **Identifier/Keyword** 状态 (继续读取字母数字, 结束后匹配关键字表)。
 - 遇到数字 -> 进入 **Number** 状态 (继续读取数字, 若遇到 . 进入 **Float** 状态)。
 - 遇到 . -> 进入 **Float** 状态 (要求后接数字)。
 - 遇到运算/分隔符 -> 进入 **Symbol** 状态 (直接返回对应 Token)。
 - 遇到空白符 -> 保持 **Initial** 状态 (跳过)。
 - 遇到其他 -> **Error** 状态。

4. 算法设计

- Token 类: 构造函数 `__init__(type, value)`。
- Lexer 类:
 - `__init__(code)`: 初始化代码、当前位置及正则表达式模式。
 - `get_next_token()`: 核心函数。使用 `re.match` 从 `pos` 开始尝试匹配。成功后更新 `pos`, 返回 Token 对象; 若匹配 `SKIP` 模式则循环; 匹配失败抛出异常。
 - `tokenize()`: 循环调用 `get_next_token` 直至 EOF, 收集所有 Token。